intro_cc_final_project

Report Content

1. Requested **screenshot** information below
2. **URL to the Web application.**

Team

- Tiep Tran - `tran2tp@mail.uc.edu`
- Jack Nguyen - `nguye2lo@mail.uc.edu`

Project Overview

In this project, our team performed analysis of **The Movie Database (TMDB) data.** This project features a medallion-style pipeline (Bronze -> Silver -> Gold) ingests movie metadata and credits, materializes typed Parquet tables, and produces a modeling-ready Gold table with budget, revenue, ROI, release month, genres, and lead director, cast, and production company. Two gradient-boosted regression models predict **box-office revenue** (log-transformed) and **audience rating**, using leakage-aware encoding for high-cardinality categories. Analysis are served on a Streamlit application with data exploration, visual analytics (genre ROI, directors, seasonality, production companies), and interactive prediction.

Technical Stack

- **Datastore** - Datalake on **AWS S3** following *medallion architecture*
  - **Bronze** - raw gzipped TMDB JSON (`discover` pages + per-movie details)
  - **Silver** - typed Parquet: `movies`, `cast`, `crew`
  - **Gold** - modeling-ready `gold_movies.parquet`
- **Interactive Web page** - Streamlit application hosted on **AWS EC2**
- **Data loading** - Data pull on Streamlit will pull fresh data from Gold folder on S3
- **ML Model Application** - 2 gradient-boosted models for predicting *user rating* and *revenue* of a hypothetical movie setup (director, budget, runtime, lead cast, ...)

Report

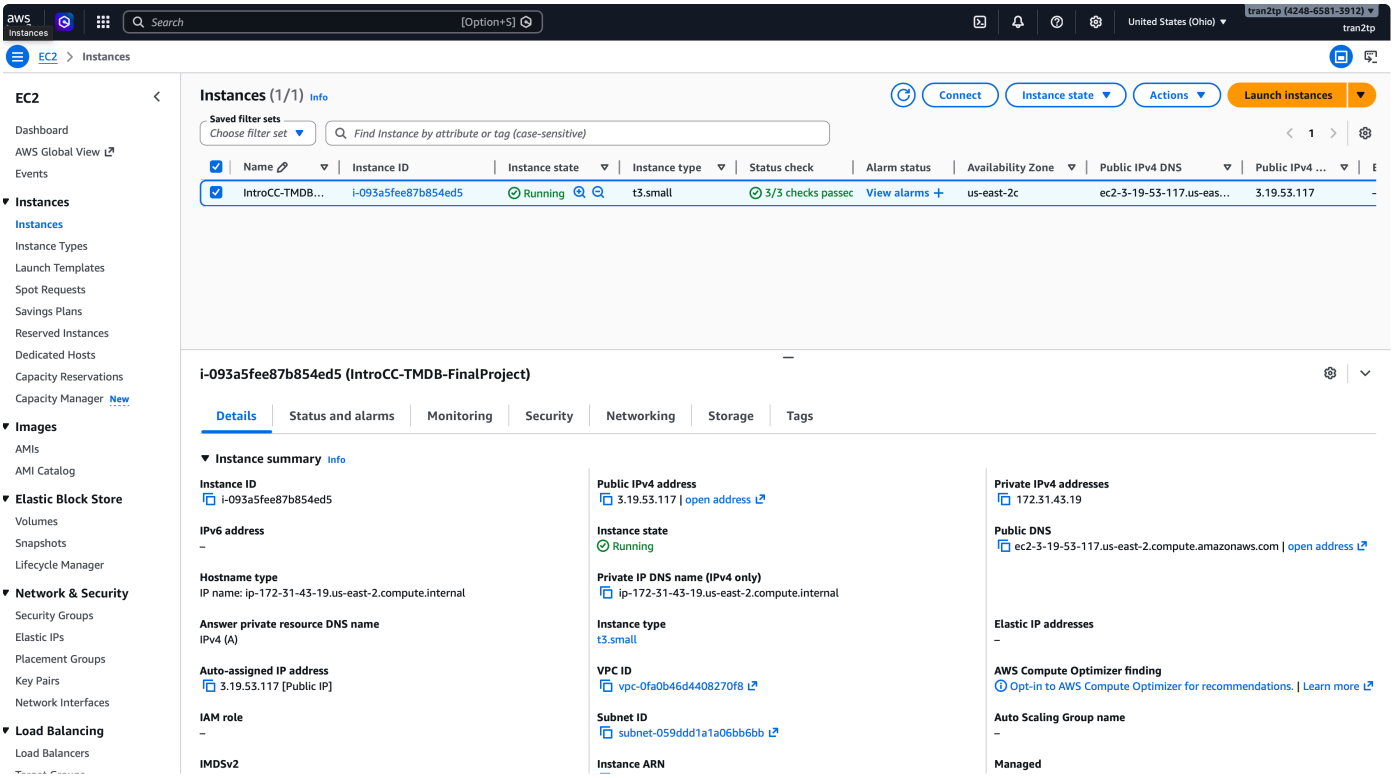
1 Write-Up on ML Models. (Point: 1)

1. **Linear Regression** - This ML model basically tries to draw a best fitting straight line that would resulted in minimal vertical distance to all of the data sample. It works on the assumption that there is a linear relationship between the input features and the target variable. It is highly interpretable, fast to train, and serves as a great baseline, but it may struggle with complex, non-linear relationships.
2. **Random Forest** - This is an ensemble decision tree ML model typically used for classification or regression. It works by aggregating the outputs of all its decision trees (taking a majority vote for classification or an average for regression). By combining multiple models (ensemble), it significantly reduces the risk of overfitting compared to a single decision tree and handles complex, high-dimensional data beautifully.
3. **Gradient Boosting** - This is an ensemble decision tree ML model. Unlike random forest, gradient boosting builds each decision tree sequentially rather than independently. Each new tree focuses entirely on correcting the errors (residuals) made by the previously built

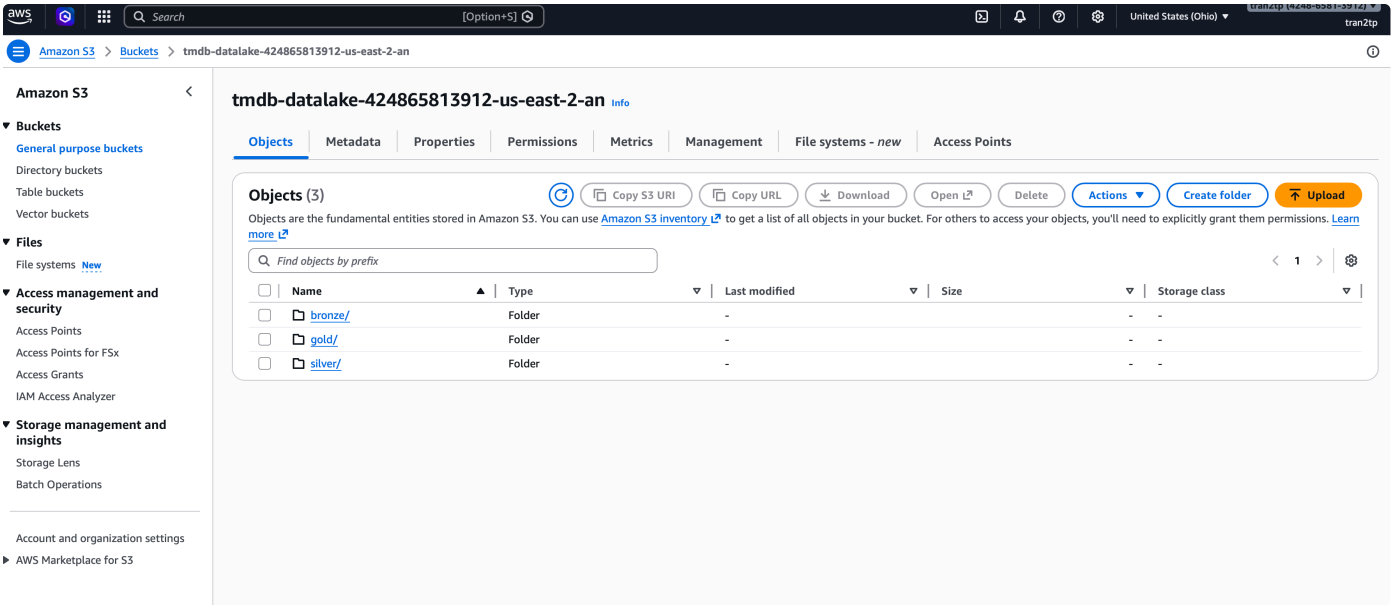
trees. This iterative improvement makes it incredibly accurate and a powerhouse for structured tabular data, though it requires careful tuning and is slower to train.

For our project, we will use **gradient boosting**: `HistGradientBoostingRegressor` – a fast gradient boosting regressor in scikit-learn, optimized for large datasets via histogram binning.

2. Web Server Setup. (Point: 2)



3. Datastore and Data Loading. (Point: 2)



4. Interactive Web Page. (Point: 2)

Home
Sample Data
Analytics
ML Prediction

Gold data

Refresh from Gold

Filters

Release year
1980 2026

Genres (any match)
Choose options

Director name contains

Budget (million USD)
0.10 489.90

Sample data — Gold

Inspect the modeling-ready table (`gold_movies.parquet`) built by the ingestion pipeline. Filters below operate on the already-ingested local parquet; re-run the CLI pipeline to refresh.

Total rows in Gold: 5,535

Schema

Filtered rows: 5,524

Columns to show

title × release_year × release_month × genres × budget_musd × revenue_musd × roi × vote_average × director_name × lead_cast_name ×

Preview row count
100

title	release_year	release_month	genres	budget_musd	revenue_musd	roi	vote_average	d
Lock, Stock and Two Smoking I	1998	8	Comedy Crime	1.35	28.3562	20.0046	8.096	G
Back to the Future	1985	7	Adventure Comedy Science Fiction	19	381.1098	19.0584	8.326	R
Predator	1987	6	Science Fiction Action Adventure Thriller	15	98.2676	5.5512	7.545	J
Snatch	2000	9	Crime Comedy	100	83.5579	-0.1644	7.82	G
Scarface	1983	12	Action Crime Drama	25	66.0233	1.6409	8.155	B
Pretty Woman	1990	3	Romance Comedy	14	463.4063	32.1004	7.448	G
The Big Lebowski	1998	3	Comedy Crime	15	47.0105	2.134	7.841	J
Match Point	2005	10	Drama Romance Thriller	15	85.6387	4.7092	7.359	W
The Untouchables	1987	6	Crime History Thriller	25	76.2705	2.0508	7.754	B
Charlie and the Chocolate Fact	2005	7	Adventure Comedy Family Fantasy	150	475	2.1667	7.038	T

Summary statistics (filtered subset)

statistic	budget_musd	revenue_musd	roi	vote_average	vote_count

5. Data Loading Web App. (Point: 2)

Home
Sample Data
Analytics
ML Prediction

Already current — Gold file unchanged.

Reload from local data/gold or S3 per DATA_BACKEND.

Refresh from Gold

Filters

Release year
1980 2026

Data analytics

Charts over the Gold modeling table (`gold_movies.parquet`). Sidebar filters apply to the chart you select below. Pick one chart at a time for b
render everything at once.

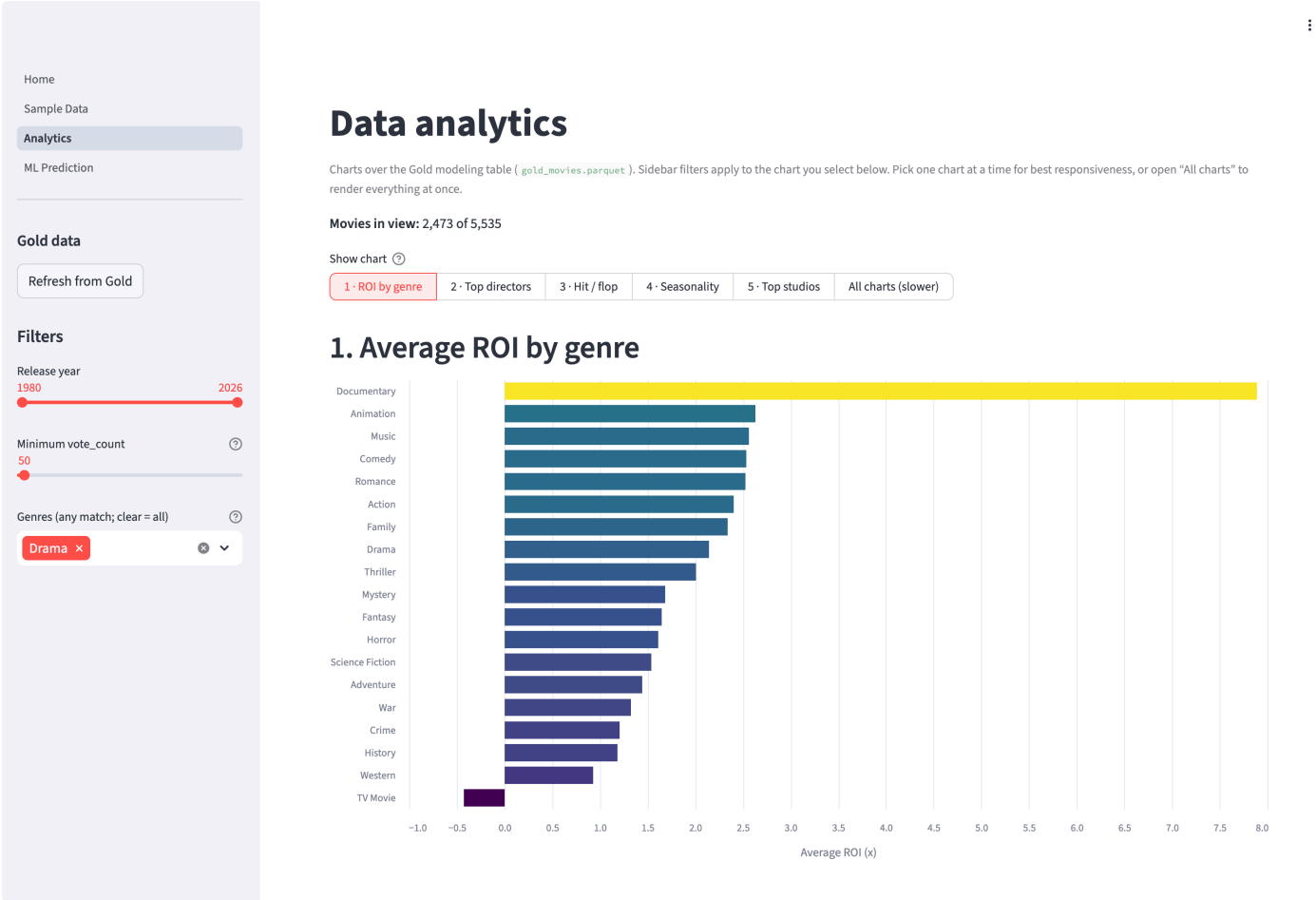
Movies in view: 2,473 of 5,535

Show chart

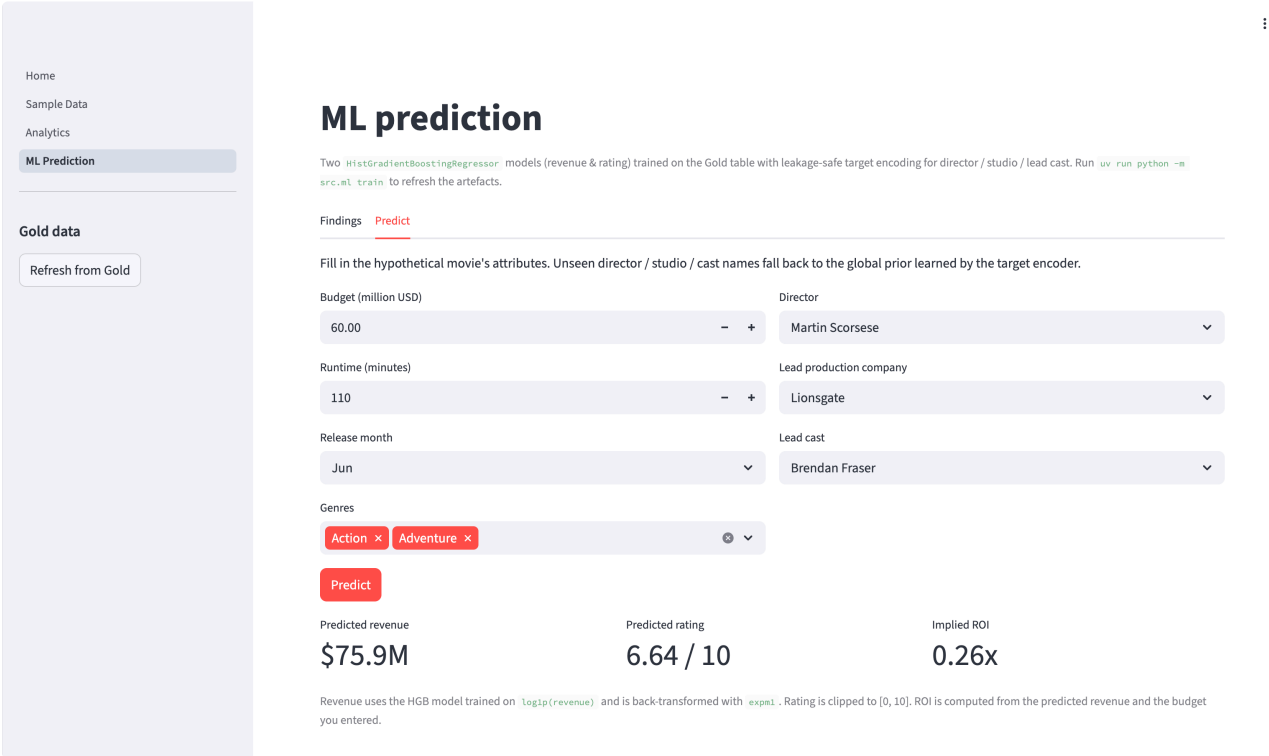
1 · ROI by genre | 2 · Top directors | 3 · Hit / flop | 4 · Seasonality | 5 · Top studios | All charts (slower)

1. Average ROI by genre

6. Web Page with Dashboard. (Point: 2)



7. ML Model Application. (Point: 2)



Home
Sample Data
Analytics
ML Prediction

Gold data

Refresh from Gold

Rating model

Target `vote_average` (0–10) with row weights `log1p(vote_count)` to down-weight noisy low-vote titles. Predictions are clipped to [0, 10].

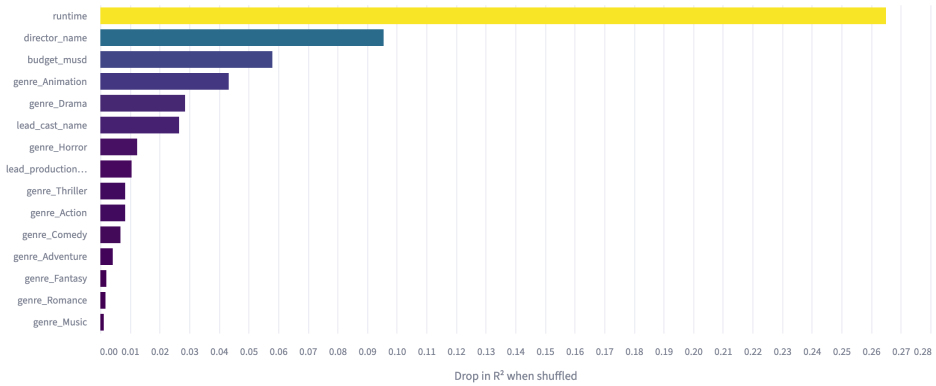
Train rows	Test rows	Holdout R^2
4,428	1,107	0.340

Metrics

Metrics are in the **training space** (log1p for revenue, native for rating). Revenue has an additional $M panel below.

Split	R^2	MAE	RMSE
Holdout (HGB)	0.340	0.453	0.577
5-fold CV (HGB)	0.312	0.447	0.569
Holdout (Ridge baseline)	0.338	0.450	0.577

Permutation importance (holdout, scoring=R^2)



Home
Sample Data
Analytics
ML Prediction

Gold data

Refresh from Gold

ML prediction

Two `HistGradientBoostingRegressor` models (revenue & rating) trained on the Gold table with leakage-safe target encoding for director / studio / lead cast. Run `uv run python -m src.ml train` to refresh the artefacts.

Findings Predict

Revenue model

Target `revenue_musd` (million USD) trained on `log1p(revenue)` so the heavy tail doesn't dominate the loss. Predictions are back-transformed to $M before reporting.

Train rows	Test rows	Holdout R^2
4,428	1,107	0.588

Metrics

Metrics are in the **training space** (log1p for revenue, native for rating). Revenue has an additional $M panel below.

Split	R^2	MAE	RMSE
Holdout (HGB)	0.588	0.856	1.080
5-fold CV (HGB)	0.559	0.848	1.092
Holdout (Ridge baseline)	0.488	0.968	1.203

Revenue back-transformed to $M:

MAE ($M)	RMSE ($M)	MAPE (revenue ≥ $1M)
69.9	141.7	1.23

Permutation importance (holdout, scoring=R^2)

ML models performance notes

Our 2 models (hypothetical movie rating prediction model and revenue prediction model) reveals the underlying patterns of movie data:

- To predict the **revenue** of a hypothetical movie, **budget** is the best indicator
- To predict the **user rating** of a hypothetical movie, **runtime** is the best indicator

The current **limitation** of these model are feature engineering - embeddings of categorial features such as Director, Lead cast, Main Production Studio. In the current model, we used **target encoding** (replacing categorical feature values with the mean of the target variable for that specific category). However, this approach also cause these features to contain less meaningful data. As a result, features like runtime and budget has the most predicting power when it comes to more logical feature such as lead cast or director.

Our team recognize this shortcoming and for future works, we want to embed these categorical features to vectors using a **sentence transformer** and switch from a simple gradient boost technique to using tabular transformer architecture like **TabPFN**.